

# Projet de Recherche Opérationnelle

Optimisation et Simulation avec Python

Yahya Sidaty El Hachmy

Matricule : C26698

Année universitaire 2025–2026

Université de Nouakchott

Faculté des Sciences et Technologies

Module : Recherche Opérationnelle

## Table des matières

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                    | <b>2</b> |
| <b>2</b> | <b>Heuristique du TSP</b>              | <b>2</b> |
| 2.1      | Description . . . . .                  | 2        |
| 2.2      | Extrait du code . . . . .              | 2        |
| 2.3      | Résultat obtenu . . . . .              | 2        |
| <b>3</b> | <b>Files d’attente hospitalières</b>   | <b>3</b> |
| 3.1      | Description . . . . .                  | 3        |
| 3.2      | Modèle mathématique . . . . .          | 3        |
| 3.3      | Extrait du code . . . . .              | 3        |
| 3.4      | Résultat obtenu . . . . .              | 3        |
| <b>4</b> | <b>Problème classique de transport</b> | <b>4</b> |
| 4.1      | Description . . . . .                  | 4        |
| 4.2      | Variables de décision . . . . .        | 4        |
| 4.3      | Fonction objectif . . . . .            | 4        |
| 4.4      | Contraintes . . . . .                  | 4        |
| 4.5      | Résultat obtenu . . . . .              | 5        |
| <b>5</b> | <b>Conclusion</b>                      | <b>5</b> |

# 1 Introduction

La Recherche Opérationnelle est une discipline qui utilise les mathématiques, l'informatique et les méthodes d'optimisation pour résoudre des problèmes complexes.

Dans ce projet, nous avons utilisé Python pour traiter trois problèmes principaux :

- le problème du voyageur de commerce avec une heuristique greedy ;
- les files d'attente hospitalières ;
- le problème classique de transport.

L'objectif général est de minimiser les distances, réduire les temps d'attente et optimiser les coûts.

## 2 Heuristique du TSP

### 2.1 Description

Le problème du voyageur de commerce consiste à trouver le chemin le plus court permettant de visiter plusieurs villes une seule fois puis de revenir au point de départ.

Dans ce projet, le fichier `heuristique_tsp.py` utilise une méthode heuristique greedy. Cette méthode choisit à chaque étape la ville la plus proche qui n'a pas encore été visitée.

### 2.2 Extrait du code

```
prochaine_ville = min(  
    non_visitees,  
    key=lambda ville: distance(ville_actuelle, ville)  
)
```

Cette instruction permet de sélectionner la ville la plus proche de la position actuelle.

### 2.3 Résultat obtenu

```
Tour trouve par heuristique greedy :  
A -> B -> E -> D -> C -> A  
Distance totale : 84
```

Ce résultat montre que l'algorithme donne rapidement une solution acceptable.

## 3 Files d'attente hospitalières

### 3.1 Description

Le fichier `hospital_queue.py` étudie les files d'attente dans un centre hospitalier.

L'objectif est d'analyser :

- le nombre moyen de patients en attente ;
- le temps moyen d'attente ;
- la stabilité du système.

### 3.2 Modèle mathématique

Le modèle utilisé est le modèle M/M/c.

Le taux d'utilisation est donné par :

$$\rho = \frac{\lambda}{c\mu}$$

avec :

- $\lambda$  : taux d'arrivée des patients ;
- $\mu$  : taux de service ;
- $c$  : nombre de serveurs.

### 3.3 Extrait du code

```
lam = 12
mu = 5
c = 3

rho = lam / (c * mu)
```

Dans notre cas :

$$\rho = \frac{12}{3 \times 5} = 0.8$$

Comme  $\rho < 1$ , le système est stable.

### 3.4 Résultat obtenu

Utilisation  $\rho = 0.8$

Nombre moyen en attente  $L_q = 2.589$

Temps moyen d'attente  $W_q = 12.94$  minutes

Temps moyen dans le systeme = 24.94 minutes

Ces résultats montrent que le système fonctionne correctement.

## 4 Problème classique de transport

### 4.1 Description

Le fichier `transport.py` traite le problème classique de transport.

L'objectif est de transporter des marchandises entre plusieurs sources et plusieurs destinations avec un coût minimal.

### 4.2 Variables de décision

Les variables  $x_{ij}$  représentent la quantité transportée de la source  $i$  vers la destination  $j$ .

```
x11 = LpVariable('x11', lowBound=0)
x12 = LpVariable('x12', lowBound=0)
x13 = LpVariable('x13', lowBound=0)

x21 = LpVariable('x21', lowBound=0)
x22 = LpVariable('x22', lowBound=0)
x23 = LpVariable('x23', lowBound=0)
```

### 4.3 Fonction objectif

La fonction objectif est :

$$\min Z = 2x_{11} + 4x_{12} + 5x_{13} + 3x_{21} + x_{22} + 7x_{23}$$

Dans le programme :

```
model += 2*x11 + 4*x12 + 5*x13 + 3*x21 + 1*x22 + 7*x23
```

### 4.4 Contraintes

Les contraintes d'offre sont :

$$x_{11} + x_{12} + x_{13} \leq 100$$

$$x_{21} + x_{22} + x_{23} \leq 150$$

Les contraintes de demande sont :

$$x_{11} + x_{21} = 80$$

$$x_{12} + x_{22} = 120$$

$$x_{13} + x_{23} = 50$$

## 4.5 Résultat obtenu

```
x11 = 80
x12 = 0
x13 = 20
x21 = 0
x22 = 120
x23 = 30

Cost = 560
```

La solution optimale permet de satisfaire toutes les demandes avec un coût total minimal de 560.

## 5 Conclusion

Ce projet nous a permis d'appliquer trois méthodes de Recherche Opérationnelle avec Python.

Nous avons étudié :

- une heuristique greedy pour le TSP ;
- un modèle M/M/c pour les files d'attente ;
- une programmation linéaire pour le transport.

L'utilisation de Python facilite la modélisation, le calcul et l'interprétation des résultats.

Ce travail montre l'importance de la Recherche Opérationnelle dans la logistique, la santé et l'optimisation des ressources.